Name : Akash Sampath

Topic : Virtualization and Containerization

Virtual Machine :

- Imagine you have one powerful physical computer known as the host. Normally it runs just one OS like Windows
- So A Virtual Machine known as VM lets you take that one physical computer and create several separate independent computers

Think of it like an apartment building :

- The physical hardware known as the server is the apartment building itself
- The Virtual Machine known as VM is an individual apartment inside the building
- The operating system like Windows or Linux running inside the VM is the tenant living in the apartment
- The software that creates and manages the VMs is called hypervisor is the property manager or landlord

Breaking Down the Key Concepts from above :

- A virtual machine tricks the operating system and applications thinking they are running on their own machine with their own CPU , Memory and Hard Drive
- So all these resources just pretend they are the software of the real hardware in the main physical computer
- If someone using it , a VM is different from a physical computer where you can install applications , run programs , save files and even crash it
- All these will not affect the main physical computer
- The operating system inside a VM is designed to think it's the only thing running.
- If you are using a VM , you get a window that acts as your own personal computer console. You click icons and it feels like the machine is just for you
- When you are using VM, it feels like you are the only person using that computer and the physical computer known as the host is actually working multiple VMs at once giving each one a tiny slice of its power
- You can run different types of software side by side and you could have a windows computer running a Linux VM and a Mac VM at the same time on the same screen
- Instead of buying 10 physical servers that only use 10 % of their power and you can buy one powerful server and run 10 VMs on it using 100 % of the hardware

Benefits and Disadvantages :

Benefits :

- Each VM lives in its own apartment (computer) and even though they are in the same building (the main physical computer) , they have their own rooms (memory) and they don’t walk into each other space (One VM cannot go interfere another VM)
- If one VM crashes or catches a virus , it doesn’t effect the main computer or other VMs. They are completely separated from each other
- Because they separated , our private data stays inside its own VM. Its like having a separate locked safe for different sets of files
- You can run Windows , Linux , Mac all at the same time on a single laptop
- Instead of buying five separate physical server, you can buy one powerful server and run five virtual ones on it. Saves cost and power on the hardware

Disadvantages :

- A VM has to ask the host (main physical computer) for permission to use the hardware.
- This extra step makes it a bit slower than a real physical computer
- If the main physical computer (the host) has a problems or security issues , it can affect the virtual machines living inside it
- Every VM needs its own full OS. If you have three VMs , you are basically storing three whole copies of Windows or Linux , which eats up a lot of your hard drive space

Virtualization :

- Think of Virtualization like a large apartment building .
- Instead of one person owning as single room house that can't possibly fill
- Therefore the building is divided into several independent apartments
- If one computer runs one operating system like Windows , Virtualization acts as a divider.
- It allows one physical computer to act like multiple smaller computers called Virtual Machines or VMs at the same time

Breaking Down the Concepts :

- Virtualization is the process of taking one physical computer and tricking it into a thinking it is actually server smaller separate computers
- Instead of buying three separate weak computers for three different tasks , you buy one powerful computer and split it. This uses the resources better
- You can move Virtual Machines from one physical server another easily because its not tied to the specific physical parts
- On one single physical laptop you could have one window running Windows , another running Linux and another running Mac all at the same time

Reasons for doing this :

- Most powerful servers only use about 10 % of their power.
- Virtualization lets them use 80 – 90 % by running many task at once
- If one Virtual Machine crashes or gets virus , the others stay perfectly safe and are separated from each other even though they share same metal and wires
- You buy fewer physical machines and they use less electricity and less space

With and Without Virtualization :

- Without Virtualization the operating system like Windows or Linux is (Either Windows or Linux) is used in the physical hardware
- Most computers are much powerful than they need to do a single task which is the problem
- This uses only 10 % of the computer power or resources like CPU and RAM and other 90 % sits idle wasting electricity and money
- If you want to run a different operating system , we have to buy a whole new physical computer
- Therefore these happens without Virtualization
- With Virtualization there is layer or software called hypervisor and this hypervisor tricks the hardware thinking it is actually running multiple independent computers
- You can run 5 – 10 or even more Virtual Machines (VMs) on that same single physical computer or server
- Instead of running one physical computer with one operating system or VM.
- You can now have more number of operating systems or VMs in one physical computer or server and you are using full 100 %
- You can run Windows in one window and Linux in another side by side on the same machine
- These don't interfere with each other and if one crashes the others keep running

Therefore ,

Without Virtualization :

- Hardware : 1 Physical Machine with 1 Operating System
- Cost : Expensive and need more hardware
- Flexibility : Tied to one system
- Safety : If one OS crashes, the server is down

With Virtualization :

- Hardware : 1 Physical Machine with Many OS
- Cost : Cheap and Can remove or add machines
- Flexibility : Can run anything anywhere
- Safety : If one VM crashes, the others stay up

A Simple Real Life Example :The Pizza :

Without Virtualization :

- You buy a whole pizza but only eat one slice. The rest goes to waste

With Virtualization :

- You slice the pizza and share it with five friends. Everyone gets exactly what they need and the whole pizza is used up efficiently

More Dept : Virtualization :

- If one computer (hardware) runs on one Operating System like Windows or MacOS
- Virtualization breaks the rule
- Now the hardware can have a physical computer with CPU and RAM
- The Hypervisor tricks different operating systems into thinking they each have their own private computer
- So you can run Windows , Linux , and another Windows setup all at the same time on the exact same laptop
- The Software doesn't need to know how the actual hardware works
- If you have 16 GB of RAM and the hypervisor can tell first virtual machine that it has 4GB and second virtual machine as 4GB
- So neither virtual machine knows the other exists and they both think they are running on their own separated or independent 4GB computer

Virtualization : Types :

Platform Virtualization :

- Runs an entire computer such as Virtual Machine
- Running Windows inside a window on your Mac using software

Resource Virtualization :

- Combining or splitting specific parts or components such as storage or networks
- In cloud storage like Google Drive we see one folder but our data is spread across so many different physical hard drivees in a giant data centre such as Google – Google Drive – Folders

Working of Virtualization :

- Imagine you have one powerful physical computer and instead of using it for just one task
- Virtualization allows you to spilt into multiple smaller copy of computers
- So each of these copied computers can run its own apps and operating system like Windows or Linux
- They don't know they are sharing the machine with others and to them its feels like they have their own private computer

Breaking Down the Working : Virtualization :

The Hardware :

- The hardware is mostly invisible to operating system
- Normally when you turn on a computer , the operating system like Windows talks directly to the physical parts like CPU , Hard Drive etc
- With Virtualization Windows no longer sees the actual physical parts. It sees an another same version of Windows
- Thinking they control Windows fully but they are just using the functionalities of Windows operating system

The Hypervisor :

- Only Hypervisor (Virtual Machine Monitor) talks to hardware
- Hypervisor is a software sits between the pretend (duplicate) computer and the real hardware
- This decides first computer should get 5 seconds of the CPU and second computer gets the next 5 secs
- Duplicate (Pretend) computer never talk to the real hardware directly
- They talk to the hypervisor and this hypervisor handles everything

Hiding :

- This refers to resource management by hiding the physical characteristics making a single physical resource appear to function as multiple resources
- You have one physical server but it looks like 10 separate servers to the people using it (One acting as many)
- You have 10 physical hard drives but the computer sees them as one single big storage drive (Many acting as one)

Real Life Example : Apartment Building :

- Physical Hardware is the large apartment building
- Without Virtualization makes one family living in the entire building wasteful
- With Virtualization makes the building divide into separate apartments
- The Hypervisor is the landlord or property manager where they manage the plumbing , electricity and structure so the tenants don't have to worry about it
- Virtual Machines are individual apartments and each tenant feels like they own the whole place but they are actually sharing the building resources like water , power with others

Levels of Virtualization :

Process Level Virtualization :

- This is small scale virtualization designed to run a single program or process as if it is created already
- You just create one app and make that one app work on different systems without changing the app itself
- So Java Virtual Machine (JVM) you write code once and it runs on Windows , Mac or Linux
- This is because Java Virtual Machine creates an uniform environment for that specific program

System Level Virtualization :

- This is large scale virtualization
- Instead of just one app , you are creating another entire computer from the present computer (inside – inside) with same hardware , operating system and all
- So we run a computer within a computer
- So it runs multiple different operating systems like Windows and Linux on the same physical machine at the same time
- You have a Windows laptop but you use system virtualization to open a windows that runs a full version of Linux inside it

Therefore ,

Process Level :

- A single program or application that Is virtualized (Virtualized What)
- Only exists while app is running (Life Span)
- Language portability like Java (Key use)

System Level :

- An entire operating system (Virtualized what)
- Runs like a separate computer (Life Span)
- Testing OS or managing server (Key use)

Depth Understanding :

- Virtualization is the process of creating a fake , software based version of something physical like a computer , storage drive or a network or OS
- Instead of buying a separate physical machine for every task you can let one powerful machine to do the work of many by splitting up

First is the Execution Environment :

- This is the place where things run
- Also where your programs or operating systems actually run
- This is splinted into two types :

Process Level :

- Imagine you bought a video game from Japan but it was designed only to run on a Japanese console. But you own a US console
- Without Virtualization the game wont run
- With Virtualization you use a compatibility layer (translator) that tricks the game into thinking its running on a Japanese Console
- Basically you are creating a virtual space (bubble) just for a single application so it can run any OS regarding the OS we planned

System Level :

- You trick one physical computer into thinking it is actually running several separate computers
- Each virtual computer can run its own operating system systems like Windows , Linux etc side by side on the same machine
- Without Virtualization only one physical server can use only one operating system
- With Virtualization Using one physical server by splitting different virtual machines with different operating systems inside each Virtual machine with having one physical server
- All the Virtual machines cant interfere other each via interfering with operating system also

Basically,

For Storage :

- Instead of having one big physical cabinet
- Virtualization chops it up into smaller virtual cabinets
- Your computer thinks it has four hard drives but actually it has just oen physical drive split into pieces

For Network :

- Instead of having to buy a dozen physical Wi-Fi routers and switches
- You can create virtual ones using software to connect all those virtual computers together

Therefore , Virtualization takes one physical thing and makes it act like many logic or virtual things to save money , space and energy

Software : Main Ways to interact with computer physical parts :

- There are two ways software can interact with a computer physical parts (the hardware
- We call these as Hosted and Bare Metal Approaches
- This is like the difference between renting a room in a house versus building your own house from scratch

Process Level : Hosted :

- Software runs inside an operating system that is already there like Windows , macOS or Android
- Imagine you want to use a specific app like a game or photoshop
- This app doesn’t talk to the computer brain (CPU) directly
- It asks Windows for permission first

Real Life :

- You are a renter and you have to follow the landlords rules and if the landlord shuts off the water and cant use the water

System Level : Bare – Metal :

- This software sits directly on the hardware and it doesn't need an existing operating system to help it
- The software is installed directly on the hard drive and takes total control of the CPU and RAM immediately and here there is no middle man
- A hypervisor used in data centres to run hundreds of servers at once

Real Life :

- You are the owner of a house and you own the land and you don't ask anyone for permission to turn on the lights. You just do it

Difference : Process Level (Hosted) : System Level : (Bare-Metal) :

Process Level :

- Needs middle man for OS
- Speed is bit slower
- Very easy to install any app
- It best for personal use or testing

System Level :

- No need middle man and can direct access to hardware
- Speed is very fast with direct access
- Very harder to set up
- Used and Best for Data Centres

Therefore ,

- Process Level is a software program running on your computer that simulates other computers
- System Level is a special software that takes over the hardware and divides into multiple independent computers

Relationship between Virtualization and Cloud Computing :

- Basically its like a relationship between a car engine and a ride sharing app like Uber
- Virtualization is the technology (engine) while cloud computing is the service built using that technology (ride)

Virtualization :

- Software that allows one physical computer to act like many small independent computers
- Instead of having 5 physical servers that are barely used , you can have 1 powerful server running 5 virtual ones
- This is like taking a large pizza and slicing it into 8 pieces and each person gets their own slice that acts like a whole meal, even though it is all coming from one pizza

Cloud Computing :

- The delivery of computing services like servers , storage , database over the internet
- Based on pay for what you use
- Based on convenience and scale where you can easily use by clicking buttons
- You don't need to own hardware you just use it on demand
- Think like renting electricity and you don't own the power plant and you just plug into the wall and pay what you use and virtualization uses cloud to give you that plug and pay experience

Basically ,

Virtualization:

- It's like creating a duplicate (pretend) computer inside your real computer
- Lets you run multiple different systems on one physical machine
- Where all are separated from each other and think like having serval virtual computers on one physical computer or server
- Virtualization is the technology that creates virtual machine on a single server

- With virtualization your virtual machine is tied to a specific physical server
- In virtualization stored data is stored on the virtual machine safely
- Virtualization lets you customize resources like CPU , RAM for your needs
- With virtualization if a VM crashes , you can try to fix it

Cloud Computing :

- Like renting computing power and storage over the internet
- You don't own the hardware you just use it on demand like paying for electricity or water
- This is a service that uses virtualization to provide resources over the internet
- Your virtual machine could be any where such as server in the cloud provider data centre which we don't know
- Storage is default for a VM is temporary and if you shut it down you lose data unless you save It else where
- Offers standard , pre-build types you can choose from
- You can start again a new VM which is faster and easier

Therefore ,

Virtualization is the tool and cloud computing is the way you use that tool to get on demand services over the internet

Virtualization : (The Tool) :

- You can create a virtual space on your own physical hardware
- The Virtual machine lives on a specific machine in your office
- We can manually decide how much RAM or CPU to give
- If it crashes you can try to fix or repair that specific VM

Cloud Computing (The Service) :

- You can access storage and programs over the internet
- The virtual machine can be anywhere in the global network
- Uses standard sizes that you can scale up or down instantly
- If it crashes you can delete it and start a brand new one in seconds

Pros and Cons of Virtualization :

Pros : (Good Things) :

- Fewer physical machines can give less electricity and so companies save on power bills
- IT staff spend less time maintaining physical servers because they manage virtual ones from the central screen
- You need fewer actual computers and so you spend less on buying and maintaining them
- Information from one virtual machine doesn't accidentally leak into another as they are sealed off
- Each Virtual machine is totally separate from the host and other VM like a private space
- You can control or change the hardware memory or storage and can share it among VMs as needed
- If one virtual machine fails, others keep running and you can quickly move them to another physical server

Cons : (Not Good Things) :

- Some older hardware or special software just cant be useful and they need to run on real physical machines
- Some servers or applications cause problems or run poorly as they are not built for virtual environment

Hypervisor :

- Think of a hypervisor as a traffic controller or a landlord for your computer hardware
- One computer has one operating system like Windows or macOS that uses all the hardware
- So hypervisor changes that by allowing one physical computer act like several independent computers
- Your computer hardware like CPU , RAM , Storage is like a single family house where only one family such as the operating system lives there
- With hypervisor that house is turned into an apartment building and hypervisor sets up the hardware so different that tenants such as multiple operating systems can live in their own private spaces at the same time without interfering each other

Basically ,

- The Host is the actual physical computer (The building)
- The Guest (or instance) are the virtual computers running inside (the apartments)
- Virtual Machine Manager known as Hypervisor controls everything you use like creating , deleting or limit the power of each virtual machine

Also ,

- Hypervisor is like a small software layer whose only jobs is to sit between the physical hardware and the operating system
- Hypervisor takes the single resources of the computer from its CPU , RAM and Storage and divides them up giving a share to each virtual machine

Host Machine :

- The actual physical computer that the hypervisor is running on and this is the foundation that holds everything up

Guest OS :

- These are the individual operating systems like Windows , Linux and macOS running inside the virtual machines
- These have no idea they are sharing with host machine and hypervisor

Virtual Machine Manager :

- Special tool that lets the person in charge easily control all the virtual machines and they can use to create new virtual machines by turning one off or move one to different Host machine

TYPE 1 : Hypervisor :

- In computer the operating system like Windows or macOS is the main (boss) of the hardware
- With type 1 hypervisor , changes happen to make system more efficient and flexible

Simple Break Down : TYPE 1 : Hypervisor :

- Software needs an operating system to talk to the hardware.
- So type 1 : hypervisor sits directly on the physical hardware and skips the middleman
- The actual physical machine like CPU , RAM , Disk are the components in the hardware
- Hypervisor is the software layer installed directly on that hardware
- Guest OS are multiple virtual computers like one running Linux and one running Windows sitting on top of the hypervisor

Why use it ? :

- Because there is no host in the operating system which makes it fast and secure
- If one Guest OS crashes or gets a virus , the others are completely safe because the Hypervisor keeps them in separate areas
- This is how cloud services like AWS or Azure work
- One giant physical server can be split into 50 smaller virtual ones for different people

In Depth : Type 1 : Hypervisors :

- Known as Bare-Metal virtualization
- Instead of one computer running one operating system like Windows
- This hypervisor lets one computer run many operating systems at the same time
- So you have a hardware and then Windows and then your apps.
- In type 1 , there is no main windows and hypervisor sits directly on the hardware making it very fast and efficient
- The hypervisor acts like a master divider and slices the CPU , RAM , Storage into different sections and each section becomes a Virtual Machine (VM)
- If virtual machine ONE crashes or gets a virus then virtual machine TWO keeps running fine
- This is remote control that lets you pick up an entire virtual machine and move it to a different physical server without even turning it off
- No need to install Windows or Linux installed on your computer first
- The hypervisor is the first thing to install
- You don’t have to install Windows just to run the software that creates other computers
- You just install the hypervisor directly to enter the hardware
- The hypervisor creates a perfect copy of a computer hardware like CPU , Memory etc for the VM
- The VM are completely isolated from each other
- The hypervisor can move resources around automatically
- The hypervisor only job is to split up the hardware

Summary :

- A type 1 hypervisor is a tiny and super software layer that sits directly on the computer hardware.
- Its job is to divide that hardware into separate VMs where you can install operating systems ensuring if one VM fails the rest VM stay running perfectly

TYPE 2 : Hypervisor :

- Think this type 2 hypervisor as an app you install on your computer to run other computers inside it
- There are different layers like hardware , Host OS and Type 2 Hypervisor and Guest OS
- In hardware is where our physical laptop or desktop with CPU , RAM etc
- In the Host OS is the main operating system you use daily like Windows 11 or macOS
- The type2 hypervisor is the software you install and Guest OS is the virtual computer running inside the software
- This is also called a hosted hypervisor is a software that lets your run multiple operating system like Windows , Linux on top of your existing operating system like Windows or macOS
- Think of it as an app or program you install on your computer and once installed this program creates virtual machines where you can install and run other operating systems
- You use Windows laptop to run Linux inside a window
- Type 2 hypervisor is an application that lets you run other operating system inside your current one

More Depth : Type 2 : Hypervisor :

- Type 2 Hypervisor which is a core part of virtualization and this allows one computer to act like many computers at once
- Imagine you have a laptop running Windows and you want to try out Linux without deleting Windows
- Install a program that lets you run Linux in a window
- This is hosted because this sits on top of your normal operation system

Layers : Type 2 : Hypervisor :

- First is the hardware which physical computer components like metal wires , and chips
- Second is the Host OS is where your main operating system is such as Windows , macOS or Linux
- The Hypervisor is the software you installed
- Guest VMs are the virtual machines or extra computers running inside the hypervisor

Some main concepts: Type 2 : Hypervisor :

- These hypervisors are not good at sharing RAM automatically
- If you tell a virtual machine to take 4 GB of RAM and it takes and doesn't like to give it back easily
- As this hypervisor sits on your main OS like Windows and if your windows gets a virus , the virtual computers inside it are also in trouble
- The foundation which is the Base OS must be working good
- This is not meant for huge data centres and its perfect for students who just want to test a new app or try different operating system on their personal laptop

Real Life Examples : Type 2 : Hypervisor :

- In simple terms this hypervisor is like you want to run multiple fake computers inside your one physical computer
- Think of a type 2 hypervisor like a regular app on your phone like a game or word
- You need to turn on your iPhone and have IOS running before you can open the game
- With this type 2 hypervisor you must turn on your main computer and start Windows or macOS first
- Only after your main computer is running you can open this special software like the hypervisor to create the fake computers inside it

Breaking Down the Concepts based on the Layers : Type 2 : Hypervisor :

- In first layer is the physical machine which is actual machine
- In second layer is the base operating system like Windows , Linux or MacOS installed on that physical machine which is the main computer you use
- The third layer is the hypervisor software like Virtual Box which is just a program running on your main Windows desktop
- The fourth layer is the virtual machines like guests and these are fake computers running inside that program

Breaking Down Concepts based on Storage : Type 2 : Hypervisor :

- Type 2 hypervisors don't support over dynamic allocation of RAM
- RAM is your computer's short term memory
- With this type of setup you can tell a virtual machine it needs 4GB of memory and that memory is reserved and taken away from your main computer
- You cannot borrow memory from elsewhere or change it easily while the machine is running
- Have to be careful if 4GB too much and will my main computer slow down or is 4GB enough or Will the virtual machine crash

Breaking Down Concepts based on Use Cases : Type 2 : Hypervisor :

- Type 2 hypervisors are typically found in environments with a small number of servers which is suitable for testing new software
- This setup is good for regular people and students

Real Life : Concepts : Based : Use Cases : Type 2 : Hypervisor :

- If you want to try a new program but you are scared it might have virus and your main Windows computer gets messed up
- When you open the hypervisor app and test it inside the virtual machine and if its breaks you can just delete that fake computer and your real computer is safe
- If you only have one or two computers in your office is fine and this is not used by big companies like Google or Amazon because it's too slow and inefficient for massive data

Paravirtualization :

- Its like a team project where everyone known they are working together rather than a Full Virtualization setup where the guest computers are kept separated

Breaking Down of Concepts : Paravirtualization :

- In standard virtualization , a guest operating system like Windows running inside Mac OS and thinks it is running on real hardware but its running virtual
- In paravirtualization the guest OS known its running on a hypervisor. Because it know it's a guest and it can talk directly to the hypervisor to get things done faster
- To make this work , the guest OS is slightly modified and its like teaching a student a shortcut so they don't have to follow the long traditional way of doing a task
- Instead of the Guest OS trying to execute a difficult command on the hardware itself, it uses a Hyper call to ask the Hypervisor to do it

Real Life : Based : Paravirtualization :

- You walk in a and tell the manager that I am a guest and the manger gives you a special remote control instead of touching the machine
- You press a button on the remote and the manager handles the machine for you and its much faster and smoother
- The employee knows you are the boss and they are trained to talk directly to the assistant instead of knocking on the wall and just shout assistant I need a pen

More Understanding of : Paravirtualization :

- The operating system knows it is inside a virtual machine and it knows it has to share
- Because the OS know its virtual the hypervisor doesn't have to waste time thinking to be a physical hardware and does not have to work like a fake hard drive or fake network just to trick the OS
- The operating system uses Hyper Call to ask the hypervisor directly
- Instead of mailing a letter which takes time you just send a text message instant

The Big Advantages : Paravirtualization :

- The quest OS and the hypervisor are working together and talking directly
- This method is much faster and it can achieve more performance as it runs fast as if it were installed directly on the computer

The Point to note : Paravirtualization :

- You have to modify the OS and you cant just take a normal version of Windows or Linux and run this way
- Operating system core has be changed to understand to make these hyper calls
- This is why open source systems like Linux were commonly used with it because developers were allowed to modify them

Therefore ,

- Paravirtualization is a smart shortcut where the guest operating system stops pretending it owns the hardware and asks the hypervisor for permission to share it and making everything run much faster

OS -Level Virtualization :

- You are not using the whole new computer and you are just partitioning the main operating systems into private separated storage or spaces
- Like an apartment building. Everyone shares the same foundation and plumbing and each tenant has their own locked door and private space

Linux KVM :

- Known as Kernal Based Virtual Machine turns the Linux system into a hypervisor that manages the virtual machines
- This uses the computer actual hardware power directly and it can run unmodified systems like You can run Windows on top of Linux without Windows knowing its inside a virtual space
- Like a car that comes with a built in truck fix and you need not buy a whole new truck , as the car was designed to handle the extra load from the start

Differences and Downsides : Para-Virtualization :

Problem 1 : Wont run on native hardware :

- The Guest OS is in para-virtualization has been modified to talk to a virtual manager like hypervisor it becomes specialized
- If you tried to take that same OS and install directly onto a regular physical laptop without the manager known as hypervisor , it would know how to function
- Its like a person who only know how to speak through a translator and they can't function where there is no translator exists or not present

Problem 2 : Different commands and system :

- There is no universal language for para-virtualization
- If first system use on set of commands and while second system uses another
- This makes it hard to move your virtual machine from one computer to another
- It's like having several different remotes for different TVs and they all do the same thing , but the buttons are all in different places

Breakdown of two types of Hypervisor : Type 1 and Type 2 :

- Basically a hypervisor in your computer jobs is to let you run multiple computers as Virtual Machines on just on single physical hardware

Type 1 : Hypervisor : Bare Metal :

- Imagine you buy a brand new computer and instead of installing Window or mac0S , you install the hypervisor directly onto the hard drive
- Actually this sits directly on the bare metal known as hardware and there is no middle man operating system
- This is very fast and powerful because it doesn't have to share resources with a background OS like Windows
- Like a landlord who lives in the building they manage and they are right there on the property making sure everything runs perfectly

Type 2 : Hypervisor : The hosted :

- At home you have already have Windows or macOS running and you download an app like Virtual Box to run another system (OS) inside it
- It is very easy to set up and great for testing new software or running an old app without losing data from your computer
- Like a landlord who lives in different city and they have to call a property such as the Host OS to get anything done which takes little time

More depth concepts : Type 1 : Hypervisor :

- This hypervisor is like an building manager who lives directly on the land.
- He builds an apartment building known as Virtual computers and manages everything himself
- This hypervisor runs directly on the hardware and there is no operating system like Windows or macOS in between
- This is independent and doesn't need to ask anyone for permission
- As its talking directly to the hardware it becomes very fast

More depth concepts : Type 2 : Hypervisor :

- This runs on top of an existing operating system like Windows or Linux and just another program you install like Chrome or Word
- This is independent and it has to ask the main operating system for hardware access like memory and CPU
- Because it has to go through the main OS first so there is little bit of slowdown

Therefore ,

- Type 1 hypervisor known as bare metal is installed directly on the computer hardware
- Type 2 hypervisor known as Hosted such as installing an program on an existing OS like Windows

So type 1 is like owning the building and type 2 is like renting a room inside someone else house

Implementation Levels of Virtualization :

Instruction set Architecture Level :

- Known as ISA
- This like a language that the computer hardware understands
- It tells the CPU what basic commands or instruction it can understand
- Simple commands or instructions such as add numbers , move data , compare values etc
- Intel x86 architecture used in most PC and ARM architecture used in mobile phones
- So the virtual machine copies the instruction set of real CPU
- The virtual machine behaves like it has its own CPU and it understands the same instructions as the real CPU
- So if the guest OS thinks that I am running on real hardware but actually it is running inside software

Real Life Example : Instruction Set Architecture Level :

- Imagine the real CPU speaks English
- Virtual machine also speaks English
- So the programs inside the virtual machine run normally

But if :

- The real CPU speaks English and the virtual machine tries to run a program written in Japanese
- Then a translator is need and that translator converts instructions and this process is called emulation

Usage of Instruction Set Architecture Level :

- We can run very old programs on modern computers (Legacy Support)
- A developer writing an app for an ARM phone while sitting at an X86 computer (Cross Development)

Working How ? :

- Imagine you have an old video game made for Nintendo which is first hardware
- But you want to play it on your PC which is the second hardware
- You use an Emulator which acts as a real time translator
- Takes the Nintendo commands and translates them into PC commands

More concepts : Instruction Set Architecture Level :

- There is something called Native Execution and Virtualization that Instruction Set Architecture Level

Real life Example : Native Execution :

- If you give this person a book written in French and they can read it easily with no need of translation

Real life Example : Virtualization : Instruction Set Architecture Level :

- If you try to give that French speaking person a book written in Japanese
- They cannot understand it and to make it work you need a translator standing between the person and the book
- The translator reads the Japanese converts it into French in real time

Therefore , the person is the your hardware and the book is your software program

Hardware Abstraction Level :

- This is a software layer that sits between the hardware like CPU , Memory and disk etc and the operating system
- This hides the complicated hardware details and give a simple interface to the operating system
- Think Hardware Abstraction Level like a translator where hardware speaks one language
- Operating System speaks another language and Hardware Abstraction Level translates between them (hardware , operating system)
- So the operating system doesn't need to know how the real hardware works
- This is also a layer of software like Hypervisor that pretends to be the hardware
- It hides that multiple computer such as Virtual Machines are fighting for the same real RAM , CPU , and hard drive
- It tells lies like when your virtual machine asks how many CPUs do I have and the Hardware Abstraction Layer says You have 2 even though the physical machine might have 64

Why do we need it ? : Hardware Abstraction Level :

- Without Hardware Abstraction Layer or Level every piece of software would need to know the specific language or details of every piece of hardware
- It helps as a universal translator between computers physical parts such as hardware and the programs that you want to use such as the software

Therefore ,

- Hardware Abstraction Level is a layer that hides real hardware details and helps the operating system work easily with the hardware real or virtual
- This makes one physical computer look like many separate computers
- This also hides the details of physical hardware and presents a clean and standard interface for Virtual Machines to use

Operating System Level Virtualization :

- This is the way to create multiple separate environment called containers inside on operating system
- Instead of installing many full operating systems we create small isolated spaces inside one OS
- To run multiple independent virtual servers on one physical computer where they all share the same operating system
- Each one thinks it's a separate machine but actually they are all using same core system

Working : Operating System Level :

- The main operating system like Linux creates separate spaces called containers
- Each container has its own files , app and settings but they all rely on the same operating system
- This makes them very lightweight and fast because no need extra software pretending to be a whole computer

Real Life Examples : Operating System Level :

- Think of one big house which is the operating system
- Inside that house you create many separated rooms called containers
- Each room works independently and has its own app and cannot disturb other rooms
- But all rooms share the same house structure such as same OS
- Instead of installing Windows , Linux etc separately like Virtual Machines
- Containers share the same OS but they stay isolated
- There are multiple separated rooms in a single house and all sharing the same kitchen and plumbing
- Each room is isolated from the others , so people in one room cant see or disturb another
- But they all use the same basic facilities and these rooms are called containers

Therefore ,

- One operating system can create many containers
- Containers act like small independent computers
- It is very fast and lightweight and uses less memory and resources
- But cannot run a different operating system inside it

Library Support Level :

- This means running program made for one system on another system without installing a full virtual machine
- This allows you to run software made for another operating system
- This is a way to make apps designed for one system like Windows run on a different system like Linux without building a fake computer inside your computer
- A library is a collection of ready made code that programs use built in tools
- Instead of changing the whole operating system we change only some system libraries so programs think they are running on a normal system

How it happens ? : Library Support Level :

- An application talks to the operating system and then talks to hardware
- So using this library support level the application talks to modified library which translates the request and then talks to the OS
- So the library acts like a translator between the application and the operating system

Real Life Examples : Library Support Level :

- You speak Tamil and Shopkeeper speaks Hindi
- You bring a translator friend
- Now yourself , the translator and shopkeeper where translator is like the modified library

Simplified Understanding : Library Support Level :

- Imagine you have a hypervisor and a guest OS like Windows or Linux trying to run inside a virtual machine
- For them to work together they need to share a set of instructions like a rulebook
- The Library Support Level simply tells you how well the Guest OS and the Hypervisor understand each other
- Actually a library is a pre-written guidebook of instructions that tells a computer how to do basic things like drawing a window on the screen or connecting to the internet
- So that every single app doesn't have to be created fully again

Real Life Example : Library Support Level :

- Imagine you are a chef (An app) trying to cook a meal (Run a task)
- Your bring your own kitchen , own tools , own recipe books and wont ask the main chef for anything
- This becomes very independent but takes a lot of time and space
- You use the head chef kitchen and their recipe books and you bring the ingredients which is faster because you are not going to carry a whole kitchen with you

User Application Level :

- This is like you use apps like Chrome , MS Word and games etc inside a virtual environment just like you use them on a normal computer
- Inside the Virtual machine you install an operating system like Windows or Linux
- Then you install applications and you use those applications normally
- This is where users run and use software applications inside a virtual machine and also where user actually interact
- This is like running a program in a protected private space so it can't interfere anything else on your computer
- It is used to run a app in an environment it wasn't originally built for
- It doesn't require a whole new OS and uses very little RAM and CPU
- The app is isolated or separated from the rest of the system
- This is also a way to run software so that it doesn't mess up your actual computer settings or conflict with other programs
- The app or software is separated or isolated from the operating system and other application

Real Life Examples : User Application Level :

- Imagine your real computer which is our house
- The virtual machine which is the room inside the house
- The Applications like Chrome , Word etc are the people working inside the room
- Imagine your computer is an apartment building
- The Hardware like CPU , RAM is the physical building itself
- The Operating system is the Windows which is the landlord who makes the rules and manages the plumbing and electricity
- Now User Application Level is like giving each tenant (application) their own personal room within their apartment

Virtualization Types : Full Virtualization :

- It allows a computer to run multiple operating systems like Windows and Linux at the same time on a single hardware
- The Guest OS is the one you are installing thinks it is running on a real physical computer
- It has no idea actually inside a virtual space

Breakdown of Concepts : Full Virtualization :

Doesn't need to modify the Guest OS :

- Normally an operating system wants to own the hardware and you have to edit the OS in other types of virtualization
- Using Full Virtualization you can take off the version of Windows or Linux and install it exactly as its and you don't need to change a single line of its code

Binary Translation : Full Virtualization :

- This the translator sitting between the OS and the hardware
- When the Guest OS tries to send a command that might crash the whole system and a middleman such as the Hypervisor catches that command and quickly translates it into a safe version and we can run it

More Depth : Full Virtualization :

- This means creating a complete virtual computer inside your real computer
- Think your computer is running normally and inside it you create another fake computer such as virtual machine
- This virtual machine can run its own operating system like Windows , Linux etc as if it is a real separate computer

No change to the main operating system (Host OS): Full Virtualization :

- The real operating system does not need to be modified and it runs normally

Uses Binary Translation :

- Some special instructions from the virtual machine cannot directly run on real hardware
- So this converts or translates those special instructions into safe ones
- This is called Binary Translation

Sensitive Instructions :

- Some instructions control hardware like CPU , Memory
- These are sensitive because they can affect the whole system
- Full virtualization catches these instructions and handles them safely

Real Life : Full Virtualization :

- You real computer is a big house
- Virtual machines are a fully built house inside that big house
- The guest inside the small house thinks it is a real house
- But actually , everything is controlled by the big house

Therefore, Full Virtualization allows you to run a complete operating system inside another operating system without changing the main system

Host based Virtualization :

- A software is installed on a normal computer like host machine and not inside special storage hardware
- You have one computer called the host
- You install virtualization software on it like VirtualBox
- Inside that computer you can create virtual machines
- Each Virtual Machine can run its own operating system like Windows or Linux
- Its also a way to run a computer within a computer using software you instal just like any other application like Chrome or Spotify

What is this Actually ? : Host Based Virtualization :

- A physical laptop is the host
- Only runs one operating system like Windows
- With host based virtualization you can install a piece of software on that Windows system
- This software acts as a layer that allows you to run guest systems like Linux or another version of Windows inside a window on your desktop
- This happens right on your main computer not on external storage device
- You have main OS on top of that you run the virtualization software
- Inside that software you run your Guest computer known as Virtual machines
- You can run your regular apps like Word on your main computer while working on completely different systems in a Virtual machine

The Advantages :Host Based Virtualization :

- You don’t have to erase your main computers operating system or change the set up
- You just install an app like Virtual Box and this software works on almost any standard computer
- Because it sits on top of your existing OS , it can work almost any computer and if your laptop can run the software , it can run the virtual machine also

Real Life Examples : Host Based Virtualization :

- Your computer is a house and the main OS is the owner of the house
- Virtualized software is like room divider
- Each virtual machine is like separate rooms where different people (OS) can live independently
- You are buying a gaming console emulator for you PC and your pc is the host and the emulator is the Virtualization layer
- The game you play is the guest OS
- You don't have to change your PC hardware to play the game , you just run the app

Full Virtualization and Para Full Virtualization : Understanding : Real Life example

In the restaurant :

Full Virtualization is like the Universal Menu :

- You go to a fancy restaurant and order a cheese burger (Your OS instruction)
- The kitchen doesn't make cheeseburgers but the waiter translates your order into a veg cheese burger that the chef understands (Binary Translation)
- You don't care how they made it and you just got your food and you didn't have to change your order style

Para Virtualization : is like the special menu :

- You go to a restaurant where the menu is specifically designed to work with kitchen
- You order item 4 because you know the chef can make it really fast (Hyper Calls)
- You had to read the menu first to know what to order (Modified OS) but because you picked something the chef is ready for , you food comes out much faster

Binary Translation with Full Virtualization :

- Full Virtualization is like language interpreter at a global summit
- Where the guest known as Guest OS speaks a specific language and the host which is the hardware speaks another
- In Binary Translation is where the interpreter listens to the guests special instructions and translates them which the host can understand easily

Containerization : Based on Apartments vs Houses : Concept :

Without Containerization :

- Its like everyone living in their own separate house
- Each house has its own foundation , plumbing and electrical system
- If you want to move to a new neighbourhood , you have to build a whole new house from scratch
- This is heavy , slow and wastes a lot of land and materials

With Containerization :

- Its like everyone living in apartments in one big building
- The building provides the shared land , foundation and main water / electricity lines (This is the host / Your physical server)
- Each apartment is self – contained and it has its own walls , furniture and mini kitchen but it shares the building main infrastructure
- If you want to move , you just pack up your furniture and no need to rebuild the whole house
- You can move into any apartment building

Basically ,

- Containerization is like those apartments and it's the way to package your app so it can run anywhere quickly
- This is also a method of packing software like your code and everything needs to run into isolated file or box called a container
- This container includes : The App which is your actual program like website or game and the dependencies are everything the app needs to run like specific version of python , java or node.js , system libraries and settings

Why containerization is go great ? :

It works on my machine ! :

- You write a python app on your Windows laptop and it works perfectly and you send it to your friend who uses Mac and it crashes because their Python version is different
- So put your app and the correct version of python into a container and send your friend the container
- As it runs exactly the same way on their Mac because the environment inside the container is same as yours

Lightweight and Fast :

- To run three different apps , you used to need three separate virtual machines each with its own full operating system like Windows or Linux
- With containers you run all three apps on the same computer sharing the computer operating systems
- They are just three different apps and this saves memory , disk space and power and start up in seconds and not minutes

Easy to Move :

- Your container can run your laptop in your company data centre or in the cloud like Amazon Web Service or Google Cloud without any changes
- It's just same cloud environment but the software is different

DOCKER : Based on Containerization :

- Docker is a tool that lets you package an application and everything it needs to run like libraries and settings into one single unit called Container
- As everything inside the container , the app will run exactly the same way on your laptop or other laptop

Working of Docker :

- Docker image is the blueprint of your applications and if want to share your app , you share the image
- Docker Container is the live running instance of an image where you can start , stop and delete them easily
- Docker Registry is where images are stored and Docker Hub is the most famous where you pull (download) images from here and push (upload)
- Dockerfile is a simple text file with a list of instructions like a script that docker uses to build an image
- Docker Volume is where if you delete a container its data disappears and volume are like external hard drive
- Data stays safe even if the container is destroyed in docker volume
- Docker compose is a tool used to run multi container apps , instead of starting a Web App and a database container separately you can use a command such as `docker-compose.yml` to start both with this one command

Example : Code : Docker Compose File :

```
version: "3.9"

services:

  db:
    image: postgres:15
    environment:
```

```
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: akashdb
    ports:
      - "5432:5432"

  app1:
    build: ./app1
    ports:
      - "5001:5000"
    depends_on:
      - db

  app2:
    build: ./app2
    ports:
      - "5002:5000"
    depends_on:
      - db
```

Explanation : Above code :

- This file is a docker compose file
- It's like a blueprint that tells your computer how to start at the same time and make sure they can talk to each other

Breakdown of the above code :

The Database : `( db )`

- This is where your data lives
- It uses a `image` which is ready made version of PostgreSQL a popular database
- This has a `environment` where the keys to the database can given like sets the username to `myuser` and the password to `mypassword`
- It has `ports` which a opens a door with (port 5432) so other programs can send and receive data

The First App : `(app1)`

- `Build` is used instead of using a ready made image and it looks inside a folder name `./app1` on your computer to see how to build this app
- `Ports` connects your computer port `5001` to apps internal port `5000`
- `Depends_on` tells the docker don't start `app1` until the `db` is ready and needs database to work

The Second App : `(app2)` :

- `Build` helps build this app using the code inside `./app2` folder
- `Ports` uses a different port on your computer `(5002)` so it doesn't crash into `app1`
- `Depends_on` just like first app it waits for the `db` to start first

To use it :

- In your terminal type `docker-compose up`
- Docker will read and build your app , start the database and link them together automatically
- Easier than starting different programs manually (Three different)

Dockerfile : Code :

```
FROM python:3.10

WORKDIR /app

COPY requirement.txt .

RUN pip install -r requirements.txt

COPY . .

CMD [“python” , “app.py”]
```

Explanation : Above : Code : Dockerfile :

- Dockerfile tells the computer exactly how to build a container that holds your app and everything it needs to run

`FROM python:3.10`

- Start with a computer that already has python 3.10 installed

`WORKDIR /app`

- Create a folder name app inside the computer and move into it
- You are telling the computer everything we do from now happens inside the `/app` folder

`COPY requirement.txt .`

- Takes the list of libraries tools from your computer and copy into the container

`RUN pip install -r requirements.txt`

- Looks at the list and install all the libraries needed

`COPY . .`

- Copy all the rest of your code files into the container

CMD [“python” , “app.py”]

- This is the final step and tells the container when you start, run my python script

Therefore, when you run this dockerfile and once it done you have a perfect image of your app that will run the same way like other computers